Document 01 — Python & Engineering Foundations

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

None. Start here.

How to Read & Practice This Document

- **What:** solid Python basics — typed functions, config loading, logging, handling errors properly.
- **Why:** every later document assumes this. Skip it, and future bugs will feel confusing when they are really a missing type hint or a swallowed error.
- **How to practice:** this page is reading material; the code lives in the exercises. Do the **Basic** ones with no notes open and the rest with notes open, then try the **Build Task** without looking at any old solution. Use the hints (**Hint 1** → **Hint 4**) only after a real try, and ask for the full solution only by saying **"Show me the solution."**

Jump to: Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

Your first AI agent needs three boring things to be right. A secret key (like `OPENAI_API_KEY`) that can never sit in code, because code is shared and pushed to git — so it lives in a `.env` file. A check that the key actually arrived, so the program stops with a clear message instead of failing five minutes later with a confusing error from someone else's server. And a record of what it did while running unattended at 3 AM, which `print()` cannot give you.

So: **type hints and custom errors** make a function fail loudly and clearly, **virtual environments** keep this project's packages separate, **`.env` files** keep secrets out of the code, and **logging** keeps the record. The **Build Task** puts all four into two small files that every later project here starts by copying.

Core Concepts (read this first — everything you need is here)

Topics on this page: Functions, types, and why they matter · Errors: a clean way to say "something specific went wrong" · Virtual environments: keeping projects separate · `.env` files: keeping secrets out of your code · Logging: better than `print()`

Functions, types, and why they matter

A **function**'s signature — `def load_config() -> Config:` — is the label on the machine: what goes in, what comes out. **Type hints** are the `int`, `str`, `Config` words in it. Without them, reading a project means reading every function body instead of every first line.

How it really works

- Python stores the hints in `__annotations__` and then **ignores them while running**. `greet(123)` runs fine although the signature says `name: str`.
- `mypy`, `pyright` or your editor read the source **without running it** and report where two files disagree, so a hint is checked before you run. A **wrong** hint is worse than none: the code still runs and the one tool that could have warned you was told a lie.
- Hints nobody checks drift within weeks — put `mypy src/` in CI.
- `X | None` says a value may be absent and forces the caller to check, instead of a later `'NoneType' object has no attribute ...`.
- A default is created **once**, when Python reads the `def`, so `def f(bag: list = [])` shares one list across every call — in agents, one agent seeing another's messages. Use `= None`, or `field(default_factory=list)`.
- Hints cannot police outside data. An HTTP body, an environment value and **anything an AI model produced** need a run-time validator (Pydantic, in Doc04 and Doc06); behind that edge plain hints are enough.
- `Protocol` is a shape, not a family: any object with `def run(self, task: AgentInput) -> AgentResult: ...` counts as an agent, so agents fit a pipeline without a shared base class.

Hint	What it promises	Use it when
`name: str`, `age: int`	Text / a whole number	Any parameter another file passes in
`-> None`	No return value; a side effect	Writing a file, sending, raising
`value: int | None`	A number **or** nothing	Optional values — the caller must check
`data: dict[str, Any]`	"Shape unknown"	Raw JSON, at the edge only
`fn: Callable[[str], str]`	A function as an argument	Callbacks, retries, tool registries

Common mistakes:

- *Mistake:* changing what a function returns but not its hint. → *Symptom:* a crash far away, `'NoneType' object has no attribute ...`. → *Fix:* re-read the signature whenever you edit a `return`.
- *Mistake:* a mutable default, `def collect(items: list[str] = []):`. → *Symptom:* the second call sees leftovers from the first. → *Fix:* `= None`, then build the list inside.

Where you'll meet it: the Basic exercise and the Build Task need full hints. Pydantic models in Doc04 and Doc06 are built from hints, and Doc06's tools use them to tell the model what to send. In Doc11 and Project 4, typed data stops one agent sending the wrong shape to the next.

Quick cheat sheet:

- Anything another file imports gets full hints; a throwaway shell line does not.
- Never `= []` or `= {}` as a default.
- Hints inside your code; a validator where untrusted data enters.

Errors: a clean way to say "something specific went wrong"

An **error** (an "exception") is the delivery driver phoning to say "house 17 does not exist on this street", instead of leaving the parcel at some door and saying nothing. A **custom error** is that call with a specific subject line: `raise MissingConfigError(f"Required environment variable is missing: {key}")`.

How it really works

- `raise` creates an error object and stops the function there; nothing after it runs.
- Python looks in the current **frame** (one running function, with its own local variables) for a `try` with a matching `except`. No match: it discards the frame, adds a line to the traceback and moves up to the caller. The traceback is that ladder of frames, ending at the `raise`.
- `finally:` runs in each frame as the error passes through, which is how files and connections still get closed. If nothing catches it, Python prints the traceback to stderr and exits with status `1`.
- Matching is by family: `except AppError:` catches `MissingConfigError` when it inherits from it. One base class per project lets the top level catch anticipated problems and let real bugs crash.
- A returned `None` can be ignored by accident and quietly become a wrong answer; an unhandled error cannot. Loud and early beats quiet and wrong.
- Never put a secret in an error message — messages reach tracebacks, logs and error-tracking services. Name the variable, never the value.
- Mark `retryable: bool` on the class: a `429` or timeout may work next time, a `400` never will, and Doc02's backoff loop reads that flag. Keep errors to a plain message string, because `multiprocessing` pickles them and extra constructor arguments often fail to unpickle.

- Python 3.11+: agents failing together under `asyncio.TaskGroup` arrive as one `ExceptionGroup`, caught with `except*`; a plain `except Exception` misses it.

Tool	What it does	Use it when	Trap
`raise MyError("msg")`	Stops with a named problem	The value cannot be used	Bare `Exception` — nobody can catch just that
`except SpecificError:`	Handles one problem	You can retry or skip	`except Exception:` swallows typos
no `except`	Lets it crash	It is a real bug	"To be safe" hides bugs for weeks
`raise ... from e`	Keeps the cause	Wrapping a library error	Without it, the cause is lost
`return {"ok": False, ...}`	Failure **as data**	An agent tool, an HTTP handler	Inside, nobody must check it

Rule of thumb: **exceptions inside your code, error values at the edges** — and in an agent loop count the failures, or a broken tool loops forever.

Common mistakes:

- *Mistake:* `except Exception: pass` around a big block. → *Symptom:* a typo or failed API call vanishes, and a wrong answer appears later with nothing in the logs. → *Fix:* every `except` names a specific error and acts on it, or goes.
- *Mistake:* `raise ConfigError(str(e))` without `from e`, or `raise e` instead of a bare `raise`. → *Symptom:* the traceback starts at your own line and the real cause is gone. → *Fix:* `... from e` when wrapping; plain `raise` when re-raising.

Where you'll meet it: the Basic and Failure handling exercises write error classes and react to each differently; the Build Task requires `MissingConfigError`. Doc02 retries timeouts, Doc06 turns a failed tool call into a message the model can read, Doc11 makes that choice for a whole agent, and Doc14 is a whole document on these tracebacks.

Quick cheat sheet:

- Catch only what you know how to handle; never a bare `except:`.
- One base class (`AppError`) splits "expected" from "real bug".
- No secret, key or token inside an error message — ever.

Virtual environments: keeping projects separate

A **virtual environment** ("venv") is one toolbox per job: a private folder holding this project's own packages. Without it, installing for one project silently changes a version another project depends on

— no error at install time, just a confusing crash later. The cycle: `python -m venv .venv`, `source .venv/bin/activate` (Windows: `.venv\Scripts\activate`), `pip install <package>`, `pip freeze > requirements.txt`.

How it really works

- `python -m venv .venv` creates `bin/`, `lib/pythonX.Y/site-packages/` and a five-line `pyvenv.cfg`. It does **not** copy Python; `bin/python` is normally a symlink to the real one.
- Running `.venv/bin/python`, the interpreter sees `pyvenv.cfg` beside it, sets `sys.prefix` to the venv and leaves `sys.base_prefix` at the real Python. **`sys.prefix != sys.base_prefix` is how Python knows it is in a venv**, and why imports come from the venv's `site-packages`. That is the whole isolation mechanism.
- `activate` starts nothing: it is a shell script that puts `.venv/bin` at the **front of `PATH`**, sets `VIRTUAL_ENV`, changes the prompt and defines `deactivate`. So `.venv/bin/python script.py` works unactivated — the reliable form for cron, CI and Docker.
- A venv cannot be moved or renamed: `bin/` and `pyvenv.cfg` hold absolute paths. Delete and recreate — nothing of yours lives inside, which is also why `.venv/` is never committed.
- `pip freeze` is a snapshot, not a specification: it records everything installed now, including packages you no longer use. The careful pattern is a hand-written `requirements.in` compiled into a pinned `requirements.txt` by `pip-compile` or `uv pip compile`.
- Pin exact versions for anything that ships (`openai==1.54.3`); `>=1.0` means your March and September builds install different code.
- Inside Docker skip the venv — the container is the isolation. Copy `requirements.txt` and install it *before* your source, so Docker caches that slow layer.
- All the agents of one app share one venv. Two agents needing incompatible versions is a sign to split them into separate *services*, not two venvs.

Situation	What to do	Why
Starting any project	`python -m venv .venv` first	The first `pip install` is where the mess starts
Two projects, different versions	One venv each, always	`openai==1.2.0` vs `2.0.0` — the problem venvs solve
A teammate or server runs it	`activate` → `pip install -r requirements.txt`	Works only if you ran `pip freeze` last time
A cron job or CI step	`/srv/app/.venv/bin/python job.py`	Cron has no shell profile; no state can go wrong
Inside Docker	No venv; install `requirements.txt`	The container is already isolated

Common mistakes:

- *Mistake:* `pip install` without activating. → *Symptom:* the install succeeds, the import fails next run, or the package lands in another project. → *Fix:* check for `(.venv)` in the prompt, or use `.venv/bin/pip install ...`.
- *Mistake:* committing `.venv/`, or renaming a folder that holds one. → *Symptom:* a huge repository that still fails for teammates; or `bad interpreter: No such file or directory`. → *Fix:* `.gitignore` it and commit `requirements.txt`; delete and recreate the venv.

Where you'll meet it: you create one here, and every later document starts with "if it is not active, `source .venv/bin/activate`". The Basic (part 2) exercise walks the full cycle. Each project ships its own `requirements.txt` — including Project 4, the first multi-agent one — and in Doc12 that same file is what your `Dockerfile` installs.

Quick cheat sheet:

- One venv per project; commit `requirements.txt`, never `.venv/`.
- `.venv/bin/python` needs no activation — use it for cron, CI and Docker.
- Never move or rename a folder holding a venv; delete and recreate.

`.env` files: keeping secrets out of your code

Your code is a recipe — copied, shared, pushed to git. Your API keys are the combination to the safe. An **environment variable** lives outside the code, in the environment the program runs in, read with `os.getenv("NAME")`; a **`.env` file** holds those values on your machine while you develop, and is never committed.

How it really works

- Every program is handed an **environment**: `NAME=value` text pairs from your shell, Docker or the CI runner. Python copies them into `os.environ` at startup, and `os.getenv("X")` is a lookup there with `None` as the default.
- The operating system knows nothing about `.env`. It is an ordinary text file that a library reads: `load_dotenv()` (from `python-dotenv`) finds it, parses each `KEY=value` line — skipping blanks and `#` comments, stripping quotes — and writes the pairs into `os.environ`.
- With no argument it walks **up** the directory tree from the calling file, so a script in a sub-folder can pick up a parent's file you had forgotten. `load_dotenv("/exact/path/.env")` removes that surprise.
- **`load_dotenv()` never overwrites a variable that already exists.** The real environment always wins; `override=True` reverses that, which is fine locally and dangerous in production.
- That is why the same code works everywhere: **in production there is usually no `.env` at all**. The platform injects the real secrets, `load_dotenv()` finds nothing and does nothing, and `os.getenv()` reads the platform's values.
- Every value is a **string** — no numbers, no booleans. `DEBUG_MODE=False` gives `"False"`, and `bool("False")` is `True`. That is the whole story behind "why is debug mode always on?".

- Module-level code runs at import, so `API_KEY = os.getenv(...)` at the top of `config.py` runs *before* `main.py` calls `load_dotenv()`. Load first, read second — inside the loader function is the safe place. (Child processes inherit `os.environ`, which is how a subprocess gets its configuration.)
- Load and validate **once**, into one frozen config object every agent shares: required keys through a helper that raises, optional ones with a default, each type converted there (`int(os.getenv("MAX_RETRIES", "3"))`) so a bad value fails at startup, not mid-job. Mark the secret `field(repr=False)` and give the object a `safe_dict()`, so nobody prints the key by accident.
- A leaked key is not fixed by a new commit; git keeps every version. Revoke it at the provider, then add `.env` to `.gitignore` **and** `.dockerignore`. `COPY .env .` bakes the secret into an image layer that deleting it later does not remove — pass values at run time with `docker run --env-file .env`.

Value or tool	Verdict	Why
API keys, passwords, webhook secrets	**In `.env`, always**	Leaking one costs real money or real data
Anything differing laptop vs production	**In `.env`**	`LOG_LEVEL=DEBUG` locally, `INFO` in production
A prompt template, a routing table	A `.py`, `.yaml` or `.json` in git	`.env` is flat: no lists, no nesting, no review
`os.getenv("KEY", "default")`	Optional settings only	A secret with a default fails later as a 401
A `_require("KEY")` helper that raises	Required settings	Fails in the first second, naming what is missing
`.env.example` (key names, fake values)	**Committed**	It tells a new teammate what to fill in

Common mistakes:

- *Mistake:* reading a variable before `load_dotenv()` runs. → *Symptom:* the key is clearly right in `.env`, but the program says it is missing. → *Fix:* call `load_dotenv()` at the very start, or inside the loader.
- *Mistake:* treating an environment value as a boolean or number. → *Symptom:* `DEBUG_MODE=False` and debug mode is still on. → *Fix:* convert in the loader — `os.getenv("DEBUG", "false").lower() in ("1", "true", "yes")`.
- *Mistake:* committing `.env`, or copying it into a Docker image. → *Symptom:* nothing, until the key is found by someone else or a scan. → *Fix:* `.gitignore` **and** `.dockerignore`, commit `.env.example`, revoke a pushed key.

Where you'll meet it: the Intermediate exercise parses a `.env` by hand so the library stops being magic, and the Edge cases exercise makes you decide what an empty value means. The Build Task is this topic plus the error topic together. In Doc04, `OpenAI()` reads the key from the environment itself, so `load_dotenv()` must run first; and in Doc11 and Project 4 one config is checked once, then shared by every agent.

Quick cheat sheet:

- `.env` is never committed; `.env.example` always is.
- `load_dotenv()` first — before any `os.getenv()`, including at module top level.
- Every environment value is a string; convert it once, in the loader.
- Never log, print or `repr()` a whole config object.

Logging: better than `print()`

`print()` is shouting across the room; logging is the shop's book. A logger writes each message with a **time**, a **severity level** and the **name of the part of the program** that wrote it, and how much detail is kept is a setting you change, not code you edit. That matters most when nobody is watching: a job at 3 AM, a server, an agent running on its own.

How it really works

- `logging.getLogger("agent.writer")` returns the **one** logger with that name, from anywhere in the program. The dots make a family: `agent.writer`'s parent is `agent`, whose parent is **root**.
- `logger.info(...)` first checks the **effective level**: with no level of its own, Python walks up the family and uses the first one it finds. Below that level the call returns immediately, building and formatting nothing — which is why leaving `logger.debug()` calls in is cheap, and why `logger.info("x=%s", x)` beats an f-string that is built even when the line is thrown away.
- If it passes, a **`LogRecord`** is built (message, arguments, level, name, time, file, line) and goes through this logger's **filters**, which can drop it or add fields — that is how a request id lands on every line. It then goes to each **handler**: `StreamHandler`, `FileHandler`, `RotatingFileHandler` (rolls over at a size, so a service cannot fill the disk).
- **Each handler has its own level.** A handler at `INFO` throws away a `DEBUG` record the logger let through — that is how one logger gives a quiet screen and a detailed file at once.
- The formatter turns the record into text. Then, unless `logger.propagate = False`, it also goes to the parent's handlers, up to root — which is how one message prints twice.
- With no handler configured anywhere, `logging.lastResort` prints `WARNING` and above to stderr, unformatted; anything below disappears silently.
- **Five reasons a line never appears**, in the order to check: logger level too high; handler level too high; no handlers at all; a filter dropped it; `propagate=False` cut it off from your handler.
- Configure logging in **exactly one place**, called once from `main()`. A library must only call `getLogger(__name__)` and log; adding handlers there hijacks every program that imports it.

Real applications use `dictConfig`, so switching production to `DEBUG` is a config change, not a deploy.

- For an agent, the log is the only record of what it did: which agent ran, which tool it called with which arguments (redacted), how long it took, how many tokens it used. Give each agent its own named logger and put `%(name)s` in the formatter, plus a request id through a `Filter` and a `ContextVar`, or a multi-agent log is unreadable.

Level	When to use it	Example
`DEBUG`	Detail only useful while troubleshooting	`logger.debug("Routes: %s", routes)`
`INFO`	Normal, important progress	`logger.info("Loaded %s rows", n)`
`WARNING`	Unusual, but there is a fallback	`logger.warning("Region column missing")`
`ERROR`	One operation failed	`logger.error("Failed to reach the API")`
`CRITICAL`	The program cannot continue	`logger.critical("Database down")`
`logger.exception(...)`	`ERROR` plus traceback — **only** in an `except`	`logger.exception("Job failed")`

Common mistakes:

- *Mistake:* `logger.debug("Config: %s", config)` — the whole object, API key included. → *Symptom:* nothing, until the log is shared or scanned and the key `.env` protected is in a file kept for months. → *Fix:* log safe fields only, and search old logs for `sk-`, `key`, `token`.
- *Mistake:* `basicConfig()` or `addHandler()` called more than once, often at import *and* in `main()`. → *Symptom:* every message printed two or three times. → *Fix:* configure once, in the entry point; guard helpers with `if not logger.handlers:`.

Where you'll meet it: the Intermediate (part 2) exercise builds a two-handler setup and the Real-world exercise wires it to your config loader — the Build Task's `get_logger()` in miniature, copied into every later project. In Doc06 and Doc07, logging each tool call is how you see what the agent tried; in Doc11 and Project 4 the result must come with a log of which agents ran and why; Doc13 turns these lines into measurable data.

Quick cheat sheet:

- `logging.getLogger(__name__)` in every module; configure handlers **once**.
- Logger level and handler level are two gates — a message must pass both.
- Duplicate lines almost always mean handlers added twice, or `propagate`.

- Never log a secret — not even at `DEBUG`.

Go Deeper (Optional)

Optional second explanations — not needed to understand the Core Concepts above.

- Python official tutorial — functions, modules, errors.
- typing module docs — type hints in depth.
- Errors tutorial — official try/except/raise.
- Real Python — Python Logging — logging vs. `print()`.
- Real Python — Virtual Environments — why venvs exist.
- Twelve-Factor App — Config — why secrets never live in code.

Practice Exercises (before the build task)

Setup: you create the virtual environment here, in `01_python_foundations/`: `python3 -m venv .venv`, `source .venv/bin/activate`, `pip install python-dotenv pydantic`. Every later document reuses it.

Where your code lives: all of it under `01_python_foundations/practice/` (`mkdir -p practice`), never loose beside this README. Exercises are grouped **by topic, not by level** — two of them share one file, each in its own labelled section — and any exercise with two or more files gets its own folder, run from inside that folder so imports and `.env` resolve the way the solutions assume.

The full file/folder layout, all exercises:

```
practice/
├── custom_errors_practice.py       Basic + Failure handling (two sections)
├── venv_setup_practice.md          Basic part 2 — notes, not a script
├── env_config_practice.py          Intermediate + Edge cases (two sections)
├── logging_practice.py             Intermediate part 2 — get_logger(name)
├── config_logging_wiring/          Real-world — its own folder
│   ├── exceptions.py               MissingConfigError
│   ├── config.py                   load_config() -> Config
│   ├── logging_setup.py            get_logger(name), copied from above
│   ├── main.py                     wires the two together
│   └── .env                        you create this; never committed
└── build_task/                     Build Task — its own folder
    ├── exceptions.py               MissingConfigError
    ├── config.py                   load_config() -> Config
    ├── logging_setup.py            get_logger(name)
    ├── test_config.py              proves the 4 Test Cases
```

```
    ├── .env.example                key names only, no real values (committed)
    └── .env                        your real values (never committed)
```

Why each script exists:

- `custom_errors_practice.py` — practice raising and catching your own exception types.
- `env_config_practice.py` — practice reading `.env` settings and deciding what "missing" means.
- `logging_practice.py` — the one function every later file calls to get a working logger.
- `config_logging_wiring/exceptions.py` — lets `main.py` catch a missing setting specifically.
- `config_logging_wiring/config.py` — fails loudly at startup if a required setting is missing.
- `config_logging_wiring/logging_setup.py` — keeps this folder self-contained, no import back into `practice/`.
- `config_logging_wiring/main.py` — this is the first 3 lines of every script from here on.
- `build_task/exceptions.py` — gives every later document a specific error to catch.
- `build_task/config.py` — the one function every later document imports for settings.
- `build_task/logging_setup.py` — the one function every later document imports for logging.
- `build_task/test_config.py` — catches a regression here before it breaks a later document.

Jump to an exercise: Basic · Basic part 2 · Intermediate · Intermediate part 2 · Real-world · Edge cases · Failure handling · Build Task

Basic — typed function + custom error

- **What:** `def set_age(age: int) -> None:` with full hints, raising your own `InvalidAgeError` when `age < 0`.
- **Why:** a signature that already tells the reader what can go wrong is the habit most beginner code is missing.
- **How to code it:** write `class InvalidAgeError(Exception): pass`, then in `set_age()` add `if age < 0: raise InvalidAgeError(f"...{age}")`. Call it inside `try` / `except InvalidAgeError as e:` and print the message. Then add a docstring and a guard clause that returns early.
- **Save as:** `practice/custom_errors_practice.py`, under a `# Basic` section (this file also holds the Failure handling exercise, in its own section).
- **Used later by:** the Real-world wiring exercise and the Build Task — you **re-write** this `class SomeError(Exception)` pattern into their `exceptions.py` as `MissingConfigError`, rather than importing this file.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Basic (part 2) — a real venv, start to finish

- **What:** the full cycle by hand — `venv` → `activate` → `pip install requests` → `pip freeze > requirements.txt` → confirm `requests` is listed with a version.
- **Why:** this is the first thing you do on every project from here on; get it into muscle memory now.
- **How to code it:** run `python -m venv .venv`, then `source .venv/bin/activate` (Windows: `.venv\Scripts\activate`), `pip install requests` and `pip freeze > requirements.txt`. Check each step before the next one — `ls`, `which python`, `pip show requests`, `wc -l requirements.txt` — and paste every command with its output into your notes.
- **Save as:** `practice/venv_setup_practice.md` — notes, not a script, since it is all shell commands: paste each command and its output as your own record. (Keep `.venv/` out of git.)
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — read `.env` by hand, then explain why not to

- **What:** parse a `.env` file's `KEY=value` lines yourself, then rewrite it in two lines with `load_dotenv()` + `os.getenv()` and say which you would ship.
- **Why:** doing it once by hand shows what the library saves you from — quoting, blanks, comments, missing files.
- **How to code it:** open the `.env` with `with open(path) as env_file:`, loop over the lines, skip blank lines and `#` comments, split each with `key, value = line.strip().split("=", 1)` and store it in a dict. Then do the same job with `load_dotenv()` and `os.getenv("KEY")`, and compare.
- **Save as:** `practice/env_config_practice.py`, under an `# Intermediate` section (this file also holds the Edge cases exercise, in its own section).
- **Used later by:** the Build Task's `load_config()`, which ships the two-line `load_dotenv()` version **re-written** into `practice/build_task/config.py`, because that one also returns a typed `Config`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate (part 2) — a logger with two output levels

- **What:** a `get_logger(name: str) -> logging.Logger` with a `StreamHandler` at `INFO` and a `FileHandler` at `DEBUG`, then one message at each level.
- **Why:** this is the setup you want in production — quiet in the terminal, detailed in the file you read after something breaks.
- **How to code it:** `logger = logging.getLogger(name)`, then add a `StreamHandler` with `setLevel(logging.INFO)` and a `FileHandler("app.log")` with `setLevel(logging.DEBUG)`, both with the same `Formatter`, and set the logger itself to `DEBUG`. Log one `debug`, `info`, `warning` and `error` line, and compare the console with `app.log`.
- **Save as:** `practice/logging_practice.py`.

- **Used later by:** the Real-world wiring exercise, which **copies** this `get_logger()` into `practice/config_logging_wiring/logging_setup.py`; the Build Task's version adds a duplicate-handler guard. Keep the name `get_logger` exactly — every later document calls it by that name.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — wire config + logging together

- **What:** a mini project whose `main.py` calls `load_config()`, then `get_logger(__name__)`, then logs one line using a value from config.
- **Why:** config and logging are almost always used together, and this is the first three lines of every script you write from here on.
- **How to code it:** in `main.py`: `config = load_config()`, then `logger = get_logger(__name__)`, then `logger.info(f"... {config.log_level}")`. Then wrap `load_config()` in `try` / `except MissingConfigError` so a missing key prints one clear line and stops, instead of a traceback.
- **Save as:** its own folder `practice/config_logging_wiring/`, with four files: `exceptions.py MissingConfigError config.py load_config() -> Config logging_setup.py get_logger(name), copied from logging_practice.py main.py wires the two together`
- `exceptions.py` — **What/Why:** lets `main.py` catch a missing setting specifically, instead of catching every Exception blindly.
- `config.py` — **What/Why:** fails loudly at startup if a required setting is missing, instead of crashing confusingly later.
- `logging_setup.py` — **What/Why:** keeps this folder self-contained, no import back into `practice/`.
- `main.py` — **What/Why:** this is the first 3 lines of every script you write from here on.
- **Run it:** `cd practice/config_logging_wiring && python main.py` — from inside the folder, so `from config import load_config` finds the file next to it.
- **Builds on:** `practice/logging_practice.py` (Intermediate part 2) — **copy** its `get_logger()` into `logging_setup.py`, same name and signature; and `practice/custom_errors_practice.py` (Basic) — the same error pattern, re-written here as `MissingConfigError`. Copies, not imports. `load_config()` is the `.env`-reading idea from Intermediate, returning a small `Config` object.
- **Used later by:** the Build Task — the same wiring, one level more serious. Get this running first and the Build Task has no surprises left in it.
- **Stuck?** Hint 1 · Hint 2 · Hint 3 · Hint 4 · Show me the solution

Edge cases — is an empty value "missing" or "valid"?

- **What:** two tricky `.env` states — a file that exists but is empty, and a key present with an empty string as its value. Test your loader against both.
- **Why:** getting this wrong silently is how a genuinely missing secret sails through as if it were fine.
- **How to code it:** make two `.env` files — one completely empty, one with `OPENAI_API_KEY=` and nothing after it — and run your loader on each. Decide what an empty string means: the strict check is `if value is None or value == "": raise ...`; the lenient one only checks `None`.
- **Save as:** `practice/env_config_practice.py`, under an `# Edge cases` section (this file also holds the Intermediate exercise, in its own section).
- **Builds on:** the `load_config()` / `require_env()` you wrote in the Intermediate section of this same file — keep both sections side by side.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure handling — three different errors, three different reactions

- **What:** three small custom error classes, one function that raises a different one depending on its input, and three separate `except` blocks — not one catch-all.
- **Why:** a blanket `except Exception` treats a typo'd input the same as a real system failure.
- **How to code it:** write three classes (`class OutOfStockError(Exception): pass` and two more), one function that raises a different one depending on its input, and a `try` with three separate `except` blocks that each react differently. Then give the three a shared base class, for the one case where the handling really is the same.
- **Save as:** `practice/custom_errors_practice.py`, under a `# Failure handling` section (this file also holds the Basic exercise, in its own section).
- **Builds on:** the Basic section of this same file — same habit, three errors instead of one. Nothing to copy: scroll up in the file you already have.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — Config & Logging Foundation

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution

Goal: a reusable `config` and `logger` setup that every later project will import.

Requirements:

- Loads required settings (at least `OPENAI_API_KEY`) from a `.env` file.
- Fails fast with a clear, custom error if something required is missing — never a fake default for a secret.
- Gives you a typed config object, not a plain dictionary, so your editor can autocomplete and catch typos.

- Has a `get_logger(name)` returning a ready-to-use logger (console handler; level set by an environment variable).

Inputs: `practice/build_task/.env` (you create it, never committed) and `practice/build_task/.env.example` with the same key names and no real values. **Outputs:** `from config import load_config`, `from logging_setup import get_logger`. **Constraints:** no secrets in code, not even in `.env.example`; no bare `except:`; full type hints everywhere.

```
01_python_foundations/practice/build_task/
├── exceptions.py        MissingConfigError
├── config.py            load_config() -> Config
├── logging_setup.py     get_logger(name)
├── test_config.py       proves all 4 Test Cases below
└── .env.example         key names only, no real values
```

- `exceptions.py` — **What/Why:** a specific error type, so callers can catch a missing setting on its own instead of catching every Exception.
- `config.py` — **What/Why:** the one function every later document calls for settings — fails loudly at startup if one is missing.
- `logging_setup.py` — **What/Why:** the one function every later document calls to get a working logger, so setup isn't repeated per file.
- `test_config.py` — **What/Why:** catches a regression here before it breaks a later document that imports `config.py` / `logging_setup.py`.
- `.env.example` — **What/Why:** shows every teammate which keys to set, without leaking real values into git.

Run it: `cd practice/build_task && python test_config.py` — from inside the folder, so `from config import load_config` finds the file next to it.

Builds on: the Real-world wiring exercise. **Copy** `practice/config_logging_wiring/` to `practice/build_task/`, then upgrade it: make `Config` a typed dataclass, add the `.env.example`, guard `get_logger()` against duplicate handlers, and replace `main.py` with `test_config.py`.

Used later by: every later document imports these two files — `practice/build_task/config.py` (`load_config()`) and `practice/build_task/logging_setup.py` (`get_logger(name)`). Doc02 is the first to say so and copies both into its own project folder. The names are fixed: rename one here and you rename it in every document that follows.

Expected Behavior

- Missing `.env`, or a missing key inside it → `MissingConfigError` naming exactly what is missing, not a generic `KeyError`.

- Valid `.env` → `load_config()` returns a working object, and a field that does not exist is caught by your type checker, not at run time.
- `get_logger("my_module")` called from two different files produces log lines you can tell apart.

Test Cases

Scenario	Expected
`.env` missing entirely	`MissingConfigError` naming the missing key (`OPENAI_API_KEY`) — the file only matters because the key ends up unset; on a server there is no `.env` and the key comes from the real environment
`.env` exists, but `OPENAI_API_KEY` is missing	`MissingConfigError` , message names the missing key
`.env` exists and is valid	`Config` object returned, `config.openai_api_key` works
`get_logger("x")` called twice with the same name	Works both times, no duplicate log lines per message
All four rows above, run in one go	`cd practice/build_task && python test_config.py` prints one line per case

Break-It / Debug Preview

- An empty `.env` file (exists, but has nothing in it).
- `.env` has the key, but its value is an empty string — is that "missing" or "valid"? Decide, and be ready to defend your answer.
- Full guided debugging happens in 14_debugging_lab.

Interview Topics Preview

- venv vs. installing globally · mutable default arguments · the error family tree · why logging beats `print()` in production · what `__init__.py` does.

Move On When

You can build a small, multi-file Python project — with logging and config — from scratch, without being told the file layout. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1** (a concept) up through **Hint 4** (a small code snippet). Ask for the full solution only if you say **"Show me the solution."**